

MEDASSISTAI - DISEASE PREDICTION SYSTEM
BACKEND DEVELOPMENT REPORT (MILESTONE 1)

submitted By:

BOYA SAI KIRAN

TEAM - 2

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Project Overview	3
objectives	4
Technologies Used	5
Backend Project Structure	6
Backend Architecture	7
Database Configuration	8
Authentication Module	9-10
JWT Authentication	11
Role-Based Authorization	12
API Testing	13
Conclusion	14

1. PROJECT OVERVIEW

Introduction

MedAssistAI is an AI-powered healthcare platform designed to assist patients and doctors through a secure digital environment. The backend of the application is developed using **FastAPI**, while **PostgreSQL** is used as the relational database. During Milestone 1, the primary objective was to establish a secure backend foundation by implementing authentication and authorization mechanisms.

My Contribution

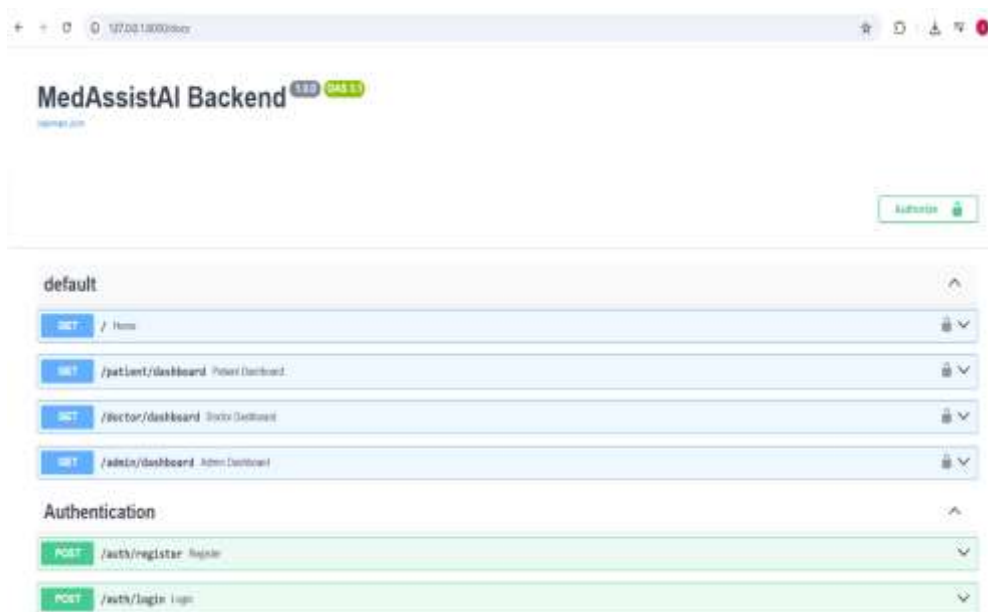
As the backend developer, my responsibilities included:

- Configuring the FastAPI backend.
- Integrating PostgreSQL using SQLAlchemy.
- Developing user registration and login APIs.
- Implementing password hashing using bcrypt.
- Generating JWT tokens for authentication.
- Protecting APIs using JWT verification.
- Implementing Role-Based Authorization for different user roles.

2. OBJECTIVES

The objectives of Milestone 1 were:

- Develop the backend using FastAPI.
- Configure PostgreSQL database connectivity.
- Implement secure user registration.
- Implement secure user login.
- Encrypt passwords before storing them.
- Generate JWT authentication tokens.
- Protect backend APIs.
- Restrict dashboard access using user roles.



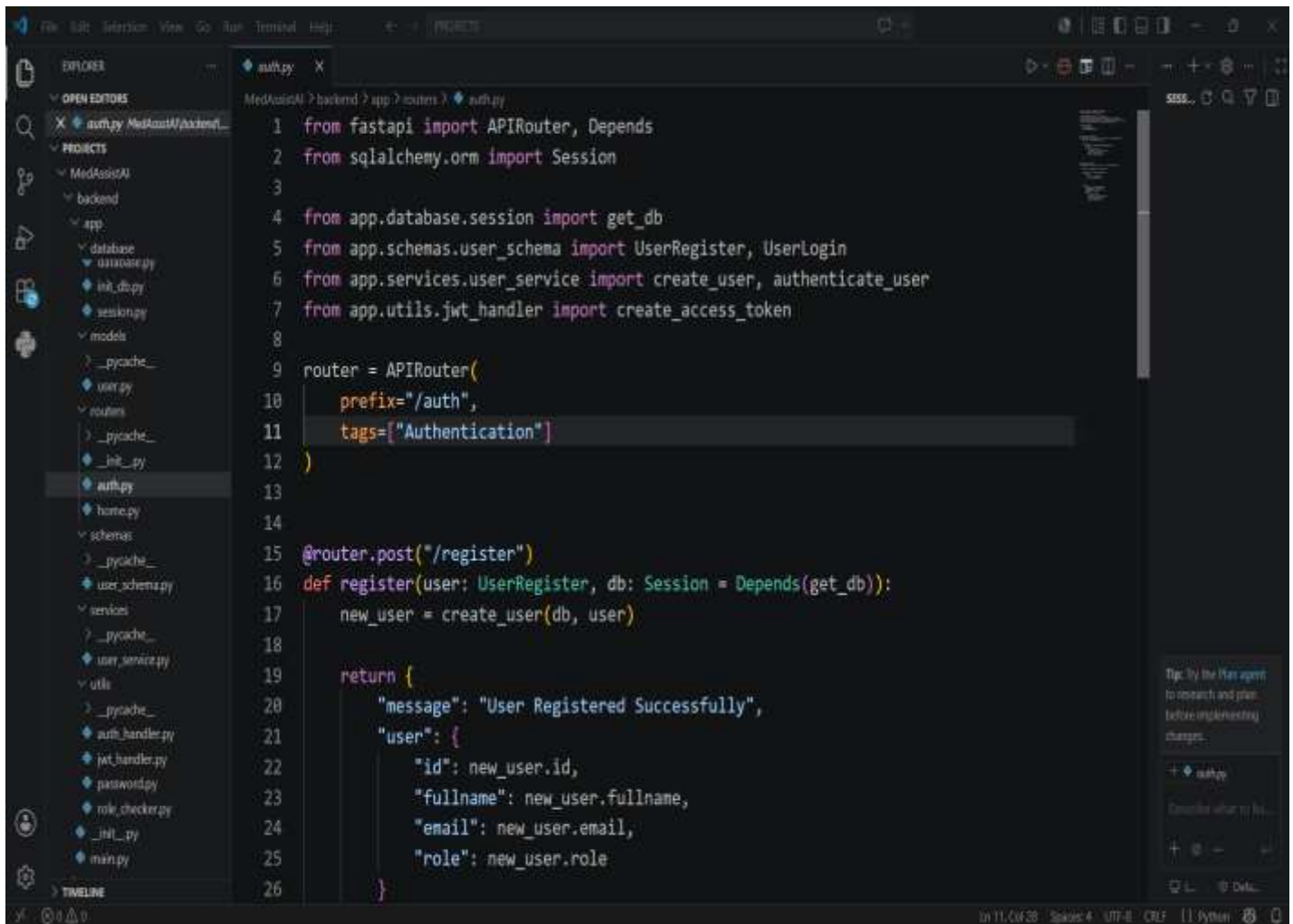
3. TECHNOLOGIES USED

Why These Technologies?

- FastAPI provides high-performance backend development.
- PostgreSQL ensures secure and reliable data storage.
- SQLAlchemy simplifies database operations.
- JWT provides secure authentication.
- Swagger enables easy API testing.

Technology	Purpose
Python 3.13	Programming Language
FastAPI	Backend Framework
PostgreSQL	Relational Database
SQLAlchemy	ORM
Pydantic	Data Validation
bcrypt	Password Hashing
python-jose	JWT Authentication
Swagger UI	API Testing
Git & GitHub	Version Control

4. BACKEND PROJECT STRUCTURE

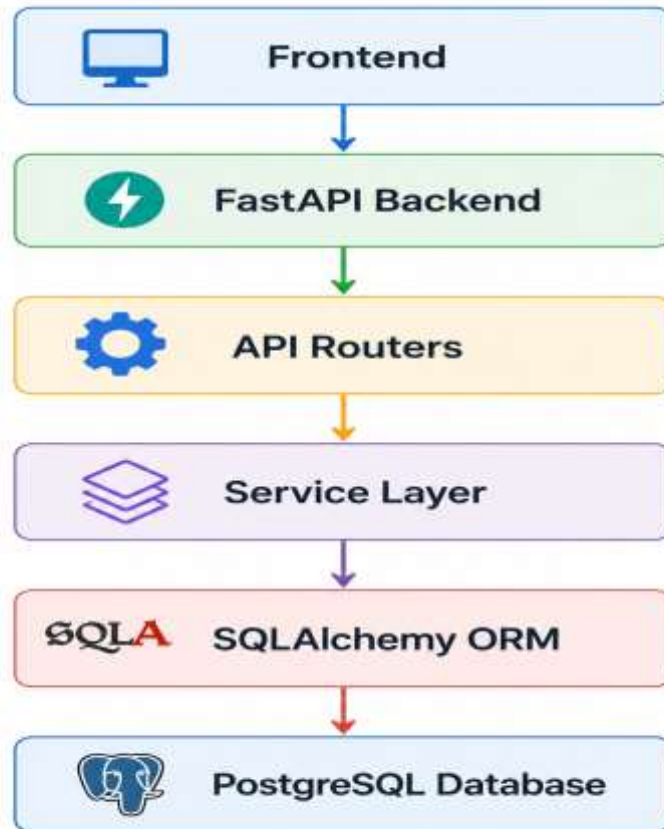


The screenshot displays the Visual Studio Code interface with a project named 'MedAssistAI'. The Explorer sidebar on the left shows the project structure, including folders for 'database', 'models', 'schemas', 'services', and 'utils'. The 'auth.py' file is selected in the Explorer and is also open in the main editor. The code in 'auth.py' defines an APIRouter for authentication, including endpoints for user registration and login. The code is as follows:

```
1 from fastapi import APIRouter, Depends
2 from sqlalchemy.orm import Session
3
4 from app.database.session import get_db
5 from app.schemas.user_schema import UserRegister, UserLogin
6 from app.services.user_service import create_user, authenticate_user
7 from app.utils.jwt_handler import create_access_token
8
9 router = APIRouter(
10     prefix="/auth",
11     tags=["Authentication"]
12 )
13
14
15 @router.post("/register")
16 def register(user: UserRegister, db: Session = Depends(get_db)):
17     new_user = create_user(db, user)
18
19     return {
20         "message": "User Registered Successfully",
21         "user": {
22             "id": new_user.id,
23             "fullname": new_user.fullname,
24             "email": new_user.email,
25             "role": new_user.role
26         }
27     }
```

The status bar at the bottom indicates the file is 'In 11, Col 28', 'Splice: 4', 'UTF-8', 'CRLF', and the editor is running 'Python'.

5. BACKEND ARCHITECTURE



Explanation

- The frontend sends API requests.
- FastAPI receives the request.
- Routers identify the appropriate endpoint.
- Services execute business logic.
- SQLAlchemy communicates with PostgreSQL.
- The response is returned to the frontend.

6. DATABASE CONFIGURATION

The project uses **PostgreSQL** as the relational database.

Why PostgreSQL?

- Open Source
- Secure
- Reliable
- High Performance
- Supports Large Data

Database Configuration

Database connection is established using SQLAlchemy

User information stored includes:

- User ID
- Full Name
- Email
- Password (Hashed)
- Role

The screenshot shows the PostgreSQL GUI interface. On the left is the 'Object Explorer' showing the database structure. The main window displays a query: `SELECT * FROM public.users ORDER BY id ASC`. Below the query, the 'Data Output' tab shows the results of the query. The table has 9 rows and 5 columns: `id`, `username`, `fullname`, `email`, and `password`. The data includes users like 'rahul', 'zhang', 'SAIKHAN', 'SAIKHAN BOYA', and 'NAM CHAMAN'.

id	username	fullname	email	password	role
9	rahul		rahul1991@gmail.com	\$2b\$12\$90M4Fgq0Pqgm...xJPe7LxjAFhaci1o8yq2v6q...	patere
10	zhang		zhangy123@gmail.com	\$2b\$12\$600uym2E4yOuf8u47zqWYUzeFvtrgpcicmUrd...	patere
11	SAIKHAN		medhase173@gmail.com	\$2b\$12\$X0xwxfvrt28uKMM0:9uWvMMFvrtgMvY9he88u...	patere
12	SAIKHAN BOYA		saikhanboyat23@gmail.com	\$2b\$12\$2629-wortu5G4FLN1FuW0K0M0YU38u54F3n.123g...	patere
13	NAM CHAMAN		medhaseyay123@gmail.c...	\$2b\$12\$B6M2Cv8yrt...8ufu5e20xvrt0t8u7N0Gv8Fv...	patere

7. AUTHENTICATION MODULE

Authentication Module

Authentication ensures that only valid users can access the system.
Two APIs are implemented.

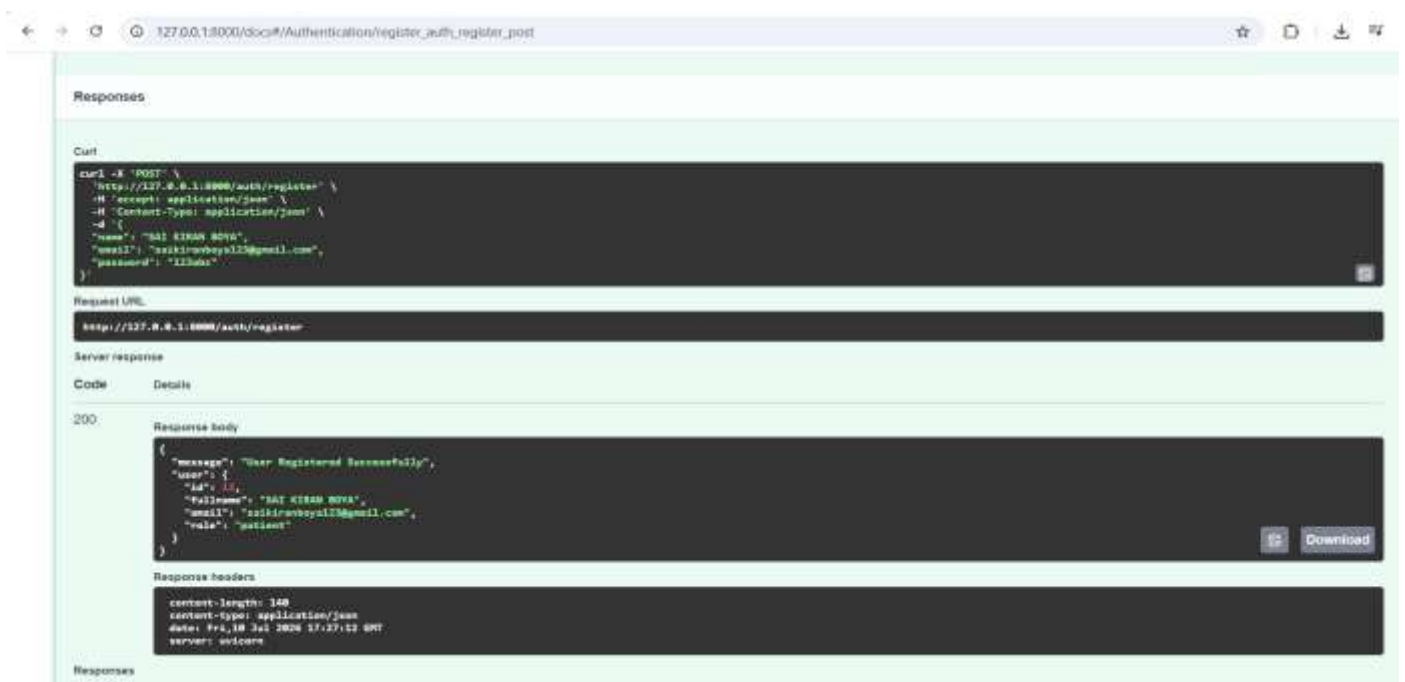
Register API

The user provides:

- Full Name
- Email
- Password

The backend performs:

- Input validation
- Duplicate email validation
- Password hashing
- Stores data in PostgreSQL



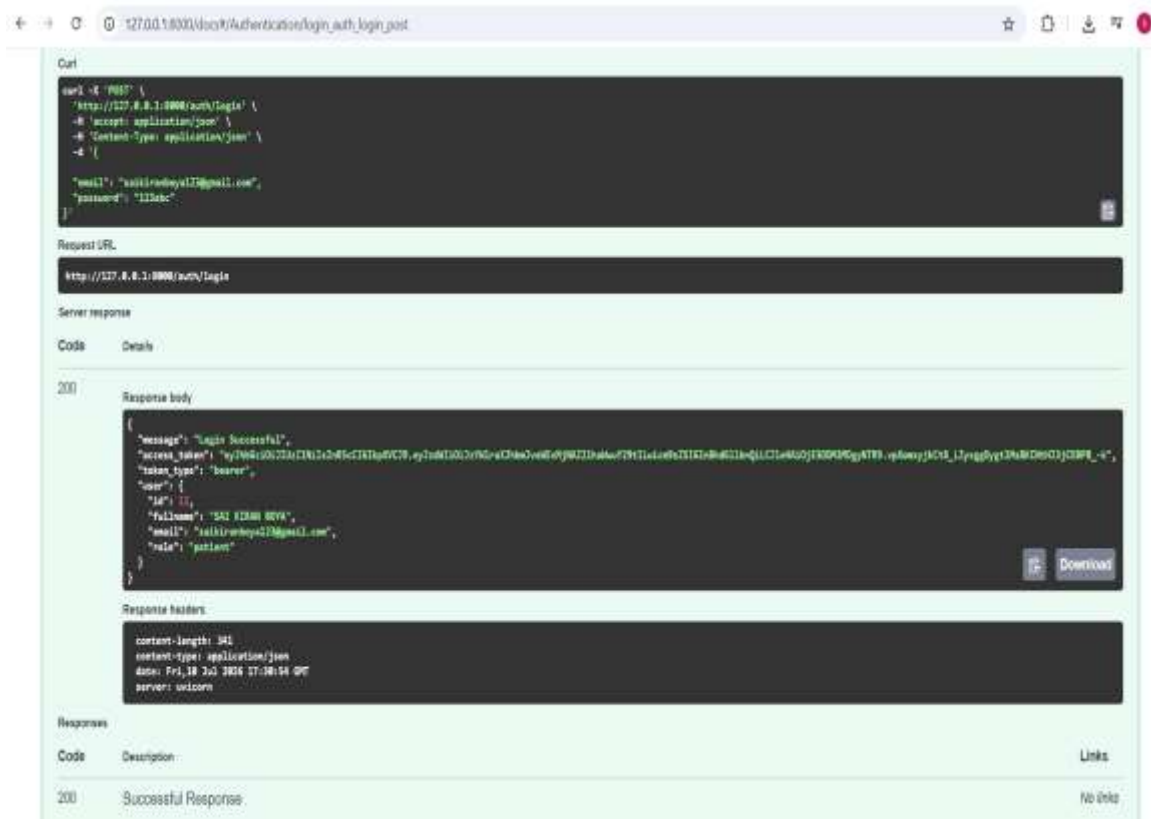
LOGIN API

The user enters:

- Email
- Password

The backend:

- Verifies credentials
- Checks hashed password
- Generates JWT Access Token
- Returns Login Success response



8. JWT AUTHENTICATION

JWT contains

- User Email
- User Role
- Expiration Time

AUTHENTICATION FLOW



9. ROLE-BASED AUTHORIZATION

Different users have different permissions.

Patient

Can access

/patient/dashboard

Doctor

Can access

/doctor/dashboard

Admin

Can access

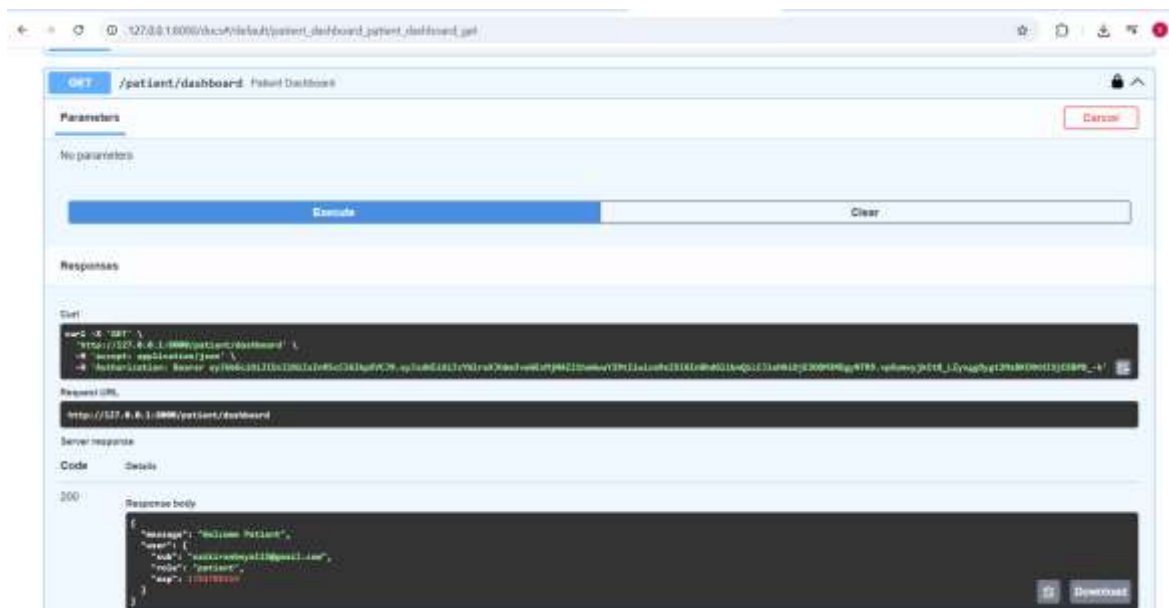
/admin/dashboard

If a Patient tries to access Doctor or Admin Dashboard,

Backend returns

HTTP 403 Forbidden

This prevents unauthorized access.



10. API TESTING

SUCCESSFULLY TESTED APIS	
METHOD	ENDPOINT
POST	/auth/register
POST	/auth/login
GET	/patient/dashboard
GET	/doctor/dashboard
GET	/admin/dashboard

Testing Results:

- Registration Successful
- Login Successful
- JWT Generated Successfully
- Patient Dashboard Accessible
- Unauthorized Access Returns HTTP 403

11. CONCLUSION

Milestone 1 successfully established the backend foundation of the MedAssistAI project.

Achievements:

- FastAPI Backend Developed
- PostgreSQL Database Integrated
- SQLAlchemy ORM Configured
- User Registration Implemented
- User Login Implemented
- JWT Authentication Implemented
- Role-Based Authorization Implemented
- APIs Successfully Tested

Future Scope:

- In the upcoming milestones, the backend will be extended by integrating:
- Patient Module
- Doctor Module
- Appointment Module
- Medical Reports Module
- AI Healthcare Features
- Frontend Integration